

End-to-End CI/CD Kubernetes Monitoring Project - Interview Guide

1. 30-Second Introduction

I built an end-to-end DevOps pipeline for a Node.js Express application. I containerized it using Docker, automated testing and image builds with GitHub Actions, deployed it to Kubernetes with Helm, enabled auto-scaling using HPA, exposed Prometheus metrics, and integrated Loki/Promtail for centralized logging. The project demonstrates the complete SDLC from code commit to production-ready deployment.

2. Architecture Flow

Developer -> GitHub -> GitHub Actions -> Docker Image -> Container Registry -> Kubernetes Deployment -> Service -> Users. Metrics go to Prometheus through ServiceMonitor. Logs go through Promtail into Loki.

3. Interview Flow

Explain: Problem, Application, Docker, CI/CD, Kubernetes, Monitoring, Load Testing, Security, Results, Improvements.

Important Questions & Model Answers

Q: Why Docker? A: To package application and dependencies consistently. Q: Why multi-stage build? A: Smaller and more secure images. Q: Why Kubernetes? A: High availability, scaling and self-healing. Q: Why HPA? A: Automatically increases pods during high CPU/memory usage. Q: Why Service? A: Provides stable networking for pods. Q: Difference between Deployment and Pod? A: Deployment manages replicas while Pod is the running instance. Q: Why Helm? A: Simplifies Kubernetes deployments with reusable templates. Q: Why Prometheus? A: Collects metrics for monitoring. Q: Why Loki? A: Centralized log aggregation. Q: What happens after git push? A: GitHub Actions runs tests, builds Docker image, pushes image, then deployment uses the new image. Q: Biggest challenge? A: Fixing CI pipeline and Prometheus endpoint integration by debugging logs and workflows. Q: Future improvements? A: Grafana dashboards, persistent storage, backup strategy, ingress, TLS and ArgoCD.